

Universal System Design Blueprint

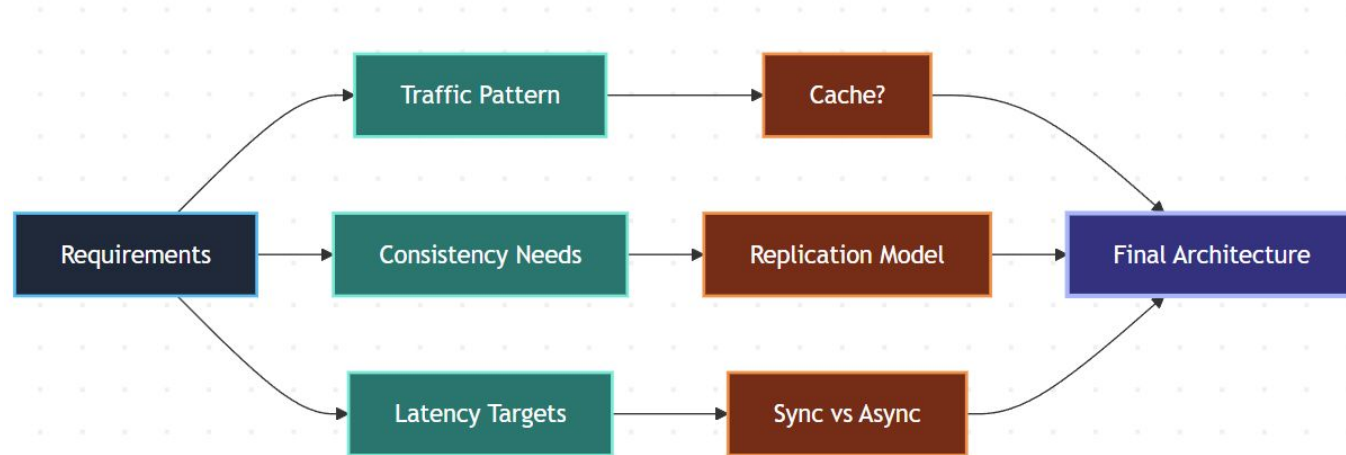
System Design Crash Course: Quick Revision Before Interviews

Rahul Rajat Singh



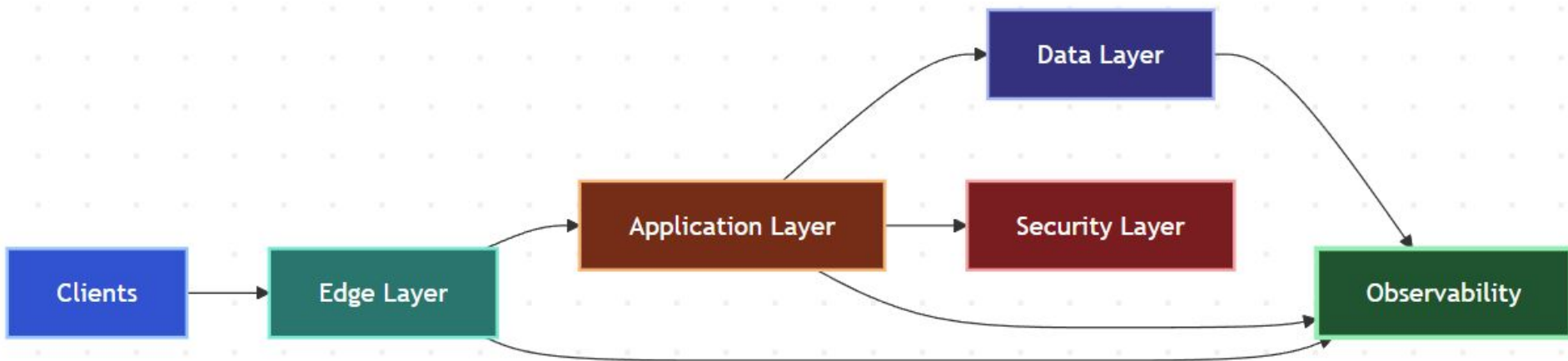
What Is a Universal System Design Blueprint

- Systems are component choices
- Same building blocks everywhere
- Requirements drive selections
- Trade-offs, not perfect designs
- Structured thinking framework



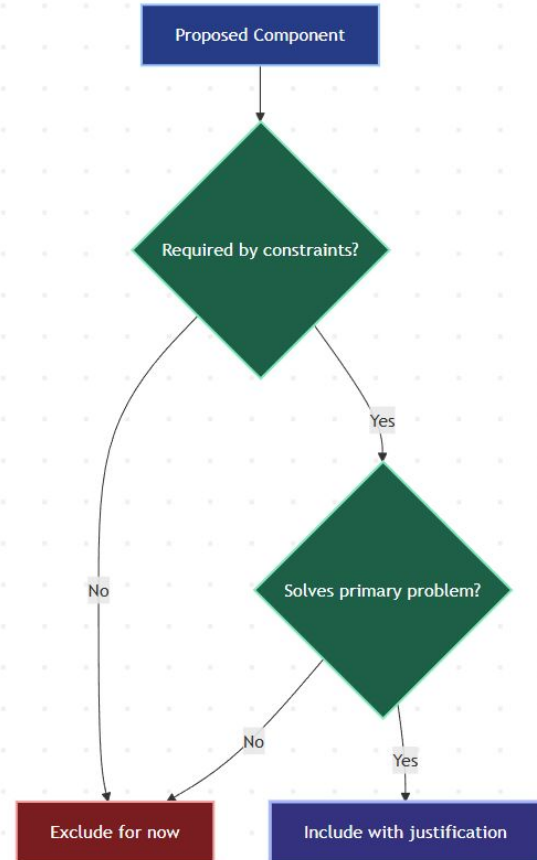
Complete System Design Component Map

- End-to-end system layers
- Clients to observability flow
- Each layer solves specific problems
- Boundaries enable scaling choices
- Missed layers cause failures



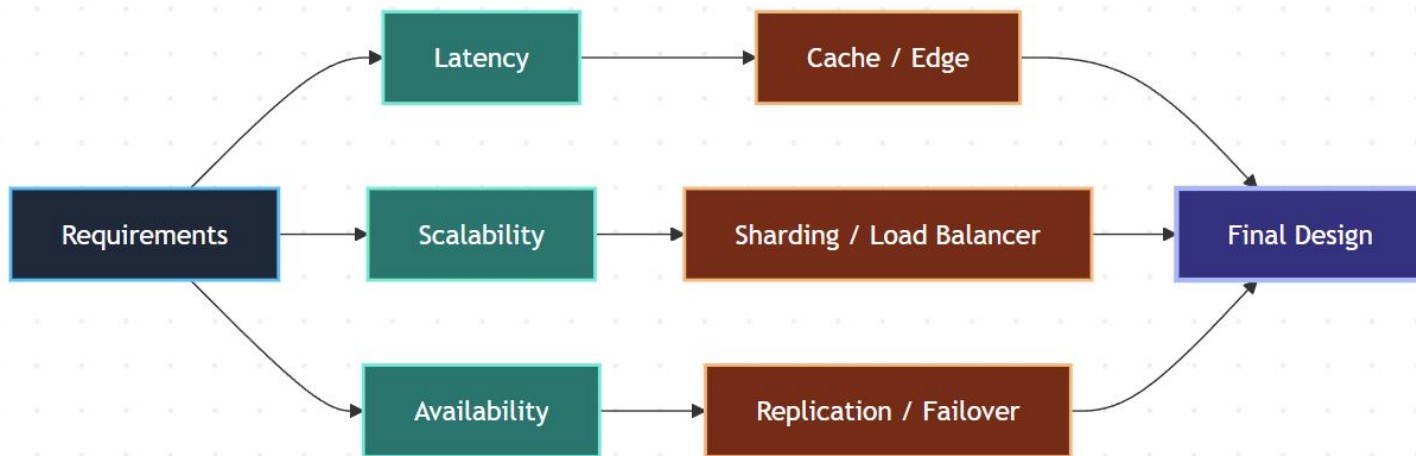
How to Decide What NOT to Include

- Default answer is no
- Add only requirement-driven components
- Complexity needs justification
- Premature scale is risk
- Simplicity improves explainability



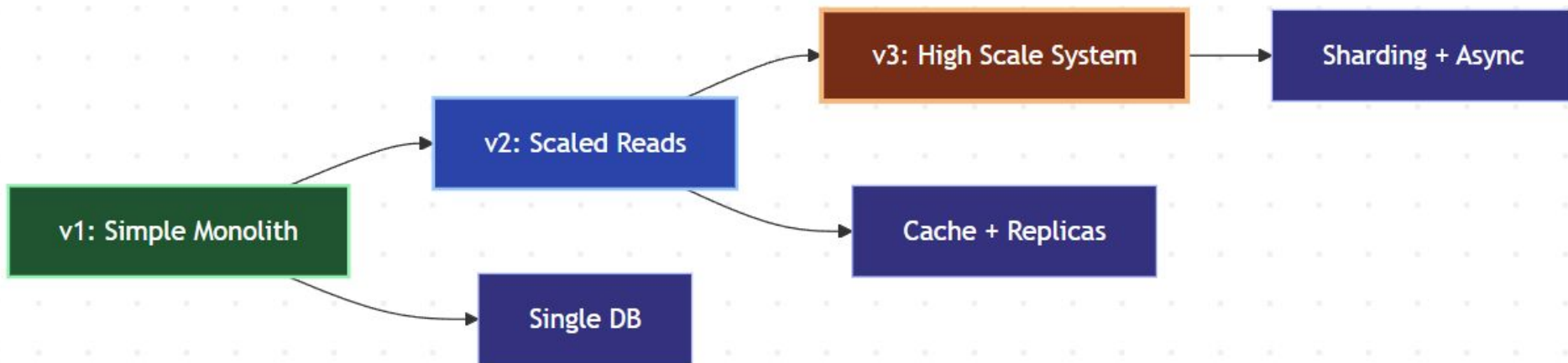
Mapping Requirements to Components

- Requirements drive architecture
- Each requirement maps to components
- No component without justification
- One requirement, many options
- Trade-offs decide final choice



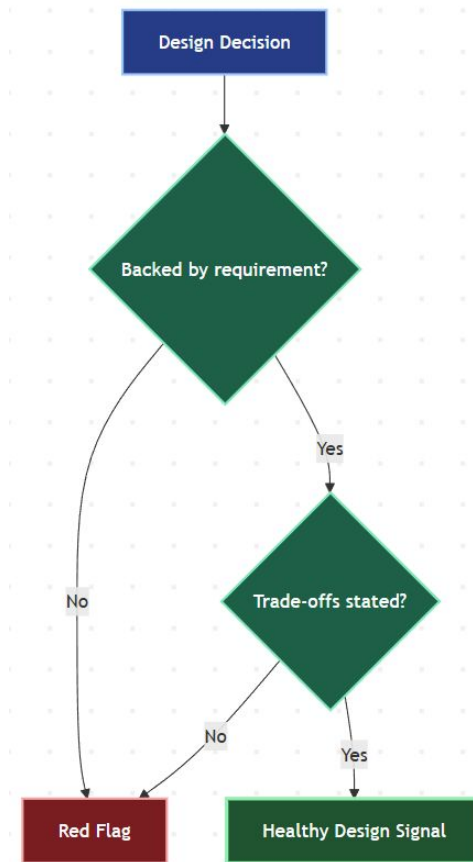
Progressive Design: Start Simple, Then Scale

- Design evolves in versions
- v1 solves core problem
- Scale added only when needed
- Complexity follows demand
- Optimize later, not upfront



Red Flags in System Design Interviews

- Over-engineering without requirements
- Ignoring trade-offs and downsides
- Jumping to final architecture
- No failure or scale thinking
- Defending choices emotionally



Estimating Scale & Identifying Bottlenecks

- Use rough, order-of-magnitude numbers
- Traffic reveals pressure points
- Find first thing that breaks
- Numbers justify architecture choices
- Precision is unnecessary



Solving System Design Problems Using the Blueprint

- Step 1: Clarify requirements
- Step 2: Estimate scale
- Step 3: Map components
- Step 4: Evolve progressively
- Same process, every interview

